

Streaming Interventions: Can Video Large Language Models Correct Mistakes as They Occur?

Apratim Bhattacharyya¹ Shweta Mahajan^{2,3*} Sanjay Haresh¹
 Rajeev Yasarla¹ Reza Pourreza¹ Litian Liu¹ Risheek Garrepalli¹
 Roland Memisevic¹

¹ Qualcomm AI Research[†] ² York University ³ Vector Institute for AI

Abstract

Learning everyday skills, like cooking a dish, relies increasingly on instructional media such as online videos. This opens the door to the use of video (and multimodal) large language models (LLMs) as task guidance assistants. A crucial capability for the real-world success of a prospective task guidance assistant is its ability to intervene proactively as soon as a mistake is apparent in order to guide the user. To evaluate this crucial capability, we introduce EGO-MC-BENCH (Mistake Corrections), a benchmark for evaluating *reactive, step-by-step* task guidance in realistic cooking scenarios. Extensive experiments show that EGO-MC-BENCH is highly challenging for state-of-the-art video LLMs. We argue that a key reason is the limited availability of training data for fine-tuning models on this task. Although there exists a wide range of cooking video datasets, existing datasets lack examples of mistakes along with appropriately timed interventions. To help address this data limitation, we also introduce EGO-COMIST, a counterfactual synthetic dataset created by transforming non-interactive cooking videos into supervised training examples showing proactive interventions. We show that fine-tuning on EGO-COMIST yields performance gains especially for smaller and more efficient video LLMs that are well suited for delivering assistance on edge devices.

1 Introduction

Traditionally, learning an everyday skill, such as preparing a new dish, requires reading a recipe or watching a video. Although being taught by domain experts, such as a chef, would be preferable, this option is typically not available or too costly. Advances in multimodal large language models (LLMs) now allow AI systems to understand and respond to speech, audio, and visual inputs in real time [27, 36, 38, 46]. This creates an opportunity to leverage such models for live, step-by-step guidance by emulating domain experts.

For multimodal and video LLMs to act as true multimodal assistants, they need to react *proactively* to events in live video streams. Consider the example in Fig. 1 (top), where a person prepares Tomato Gnocchi. The recipe calls for two teaspoons of salt. The model should perceive that only one teaspoon has been added, infer from the video that the user does not intend to add more, and proactively prompt them to add the second teaspoon. Once the user does so, the model should detect and signal completion of this step. However, current benchmarks in this area [5, 41, 43] either test a limited subset of conversational abilities or re-purpose non-reactive data—providing only a partial assessment of proactive assistance capabilities of multimodal and video LLMs.

Existing attempts to build assistants for proactive task guidance in the real world have been met with limited success [5, 51]. This is in spite of the fact that there are many large scale datasets in the

*Work done while employed at Qualcomm AI Research.

[†]Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc.

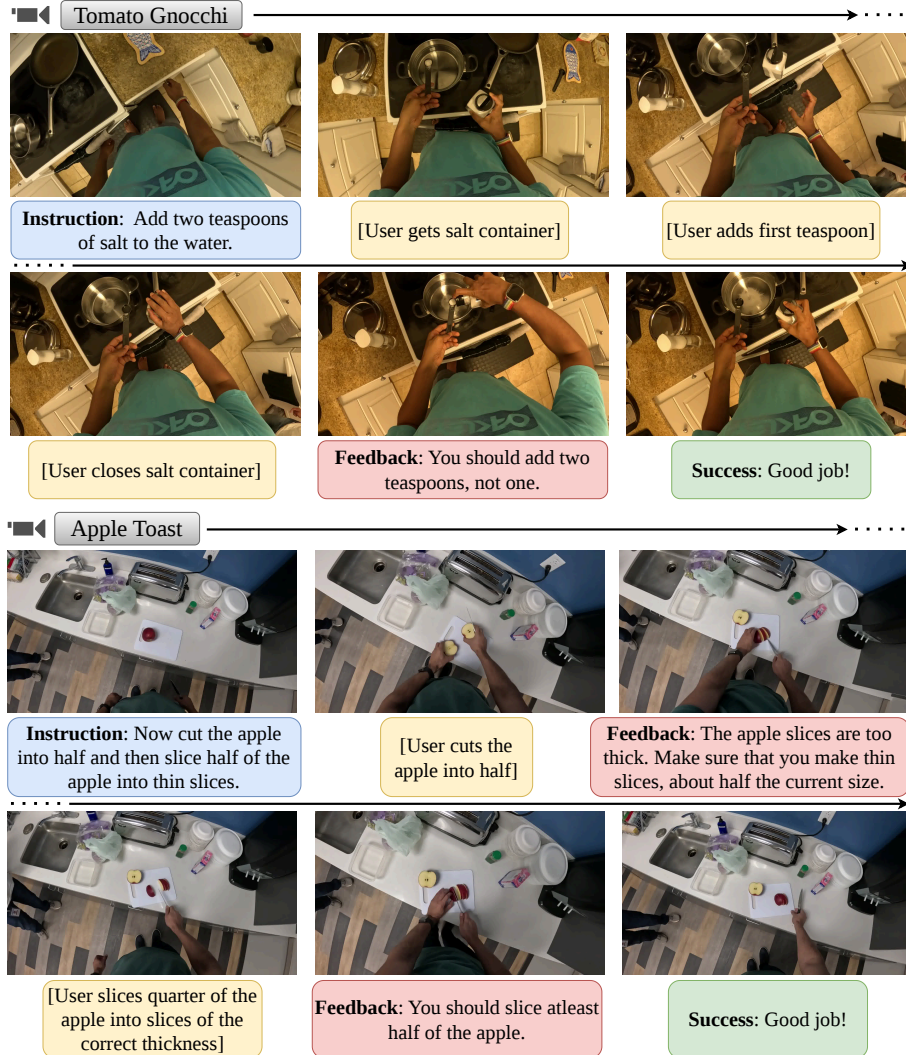


Figure 1: Our EGO-MC-BENCH: interventions with appropriate feedback whenever a mistake is apparent, guiding the user towards successful goal completion across recipe steps.

cooking domain [8, 20, 30, 52]. The key challenge is that such datasets lack suitable demonstrations of mistakes and corresponding interventions and feedbacks. Furthermore, collecting such high-quality supervision at scale is prohibitively expensive.

In this work, we introduce a novel benchmark and data generation pipeline to help address these issues. Our contributions in detail are, 1. We propose the Qualcomm Interactive Cooking: EGO-MC-BENCH (Mistake Corrections)[‡], the first benchmark to evaluate the step-by-step task guidance capabilities of multimodal LLMs in a truly reactive setup. EGO-MC-BENCH evaluates the ability of a model to intervene as soon as a mistake is apparent by providing appropriate feedback and thus guiding users to successful task completion. 2. We propose Qualcomm Interactive Cooking: EGO-COMIST, a novel synthetic dataset created by re-purposing existing (non-interactive) datasets towards supervised training on proactive, streaming task guidance. Overall, this represents a significant expansion of the Qualcomm Interactive Cooking Dataset [5]. 3. We present extensive experiments which demonstrate that the interactive task guidance task in our EGO-MC-BENCH is highly challenging for state of the art video LLMs. 4. We show that fine-tuning a model on the EGO-COMIST dataset leads to significant improvements in successful mistake interventions, especially for efficient video LLMs well suited for edge-deployment.

[‡]Qualcomm Interactive Cooking: EGO-MC-BENCH – available at this URL and EGO-COMIST – available at this URL

Table 1: Here, *Multi-step Goal Driven* refers to whether the videos are driven by a specific goal (e.g., cooking a recipe); *Step-by-Step Instructions*: whether the videos contain participants following a set of step-by-step instructions; *Timed Feedback*: whether the participants receive timed feedback per step that is sufficient to guide the subjects to the goal; *Reactive Participants*: whether the participants react to the feedback and correct their actions to successfully complete the complex multi-step goal.

Dataset	Domain	Multi-step Goal Driven	Step-by-Step Instructions	Timed Feedback	Reactive Participants	Length (hrs)
Assembly-101 [32]	Toy Assembly	✓	×	×	×	513
HowTo100M [25]	Diverse	✓	✓	×	×	134k
COIN [35]	Diverse	✓	✓	×	×	512
YouCookv2 [52]	Cooking	✓	✓	×	×	176
WTAG [4]	Cooking	✓	✓	×	×	10
HoloAssist [41]	Obj. manip.	×	✓	✓	✓	166
QEVD [28]	Fitness	×	✓	✓	✓	474
QICD [5]	Cooking	✓	✓	✓	×	94
EGO-COMIST (Ours)	Cooking	✓	✓	✓	×	124
EGO-MC-BENCH (Ours)	Cooking	✓	✓	✓	✓	10

2 Related Work

Multimodal and Video LLMs. Vision–language models have advanced rapidly on the heels of breakthroughs in large-scale language modeling. Early work such as Flamingo [2] paired a pretrained language model with learnable vision adapters, aligning visual and linguistic representations to enable strong zero- and few-shot performance. This paradigm has since yielded a diverse family of large multimodal language models [1, 3, 13, 18, 37–39] with strong results on video understanding. Progress has been further catalyzed by broad evaluation suites such as Video-MME [16], Video-MMMU [21], and LongVideoBench [44]. However, most existing models and benchmarks remain largely *turn-based*: responses only when explicitly prompted instead of proactively monitoring, or reacting to, the unfolding events in the stream. In contrast, our EGO-MC-BENCH benchmark and EGO-COMIST synthetic data generation pipelines are designed for *interactive* scenarios, where a model must track events in its input video stream and respond without explicit user prompts.

Streaming Video LLMs. Standard video LLMs struggle with the temporal demands of streaming inputs, motivating recent models that respond interactively to live video. A foundational effort is VideoLLM-online [6], trained to narrate egocentric streams. Building on this, works such as StreamMind [11], Flash-VStream [50], LiveVLM [26], and ReKV [10] propose more efficient architectures for the streaming narration setting. StreamingVLM [47] extends narration to hour-long videos by using attention sinks, while LION-FS [22] adopts a two-stream fast–slow vision encoder to improve visual grounding and accuracy. A different interaction scenario is considered by ViSpeak [17], which enables models to respond to gestures and body language in real time. In contrast to narration or QA, we study the *streaming intervention* task: detecting both mistakes and step completions within a procedural activity and intervene at the right moment, which requires fine-grained tracking of progress toward the target goal.

Benchmarks and Datasets for Streaming Video. VideoLLM-online [6] introduced an Ego4D-based narration dataset instrumental in the development of streaming video LLMs. LiveCC [7] contributed ASR-aligned datasets for live sports commentary. ProactiveQA [51] provides a streaming dialogue and question-answering dataset over egocentric videos. SVBench [48], OmniNMI [42], Ovo-Bench [23], and Streamo [45] target streaming dialogue and QA over general, real-world videos. By contrast, datasets such as HoloAssist [41], WTAG [4], and QEVD [29] are designed for interactive assistance that delivers live feedback in daily-life, cooking, and fitness scenarios. However, unlike our EGO-MC-BENCH benchmark, these datasets do not provide multi-step feedback that guides users toward successful completion of complex goals as shown in Tab. 1. Finally, the base Qualcomm Interactive Cooking Dataset [5] (QICD) frames step-by-step task guidance in a non-reactive setting. In contrast, EGO-MC-BENCH is explicitly *reactive*, evaluating interventions to mistakes as they occur.

3 EGO-MC-BENCH

We introduce Qualcomm Interactive Cooking: EGO-MC-BENCH (Fig. 1), a benchmark for evaluating multi-modal assistants that guide users *step by step* through cooking tasks with multi-step recipes. EGO-MC-BENCH targets two core capabilities: (i) detecting step completion and (ii) intervening on mistakes with timely, corrective feedback (we define mistakes as *actions that directly impede recipe completion*). Prior step-by-step benchmarks [5] are derived from non-interactive datasets (e.g., CaptainCook4D [30]), producing non-reactive scenarios where users cannot respond to feedback. In contrast, EGO-MC-BENCH models a realistic interactive setting: users react to feedback, and the assistant actively guides them toward successful completion.

Benchmark Collection. The EGO-MC-BENCH benchmark is recorded in an interactive live setup. The recording is performed using a head mounted camera in a kitchen. An instructor provides step by step instructions and feedback. The step by step instructions are recipe steps of varying complexity (Fig. 1). The instructor is positioned behind the participant as shown in Fig. 2 such that the instructor can observe the user actions in detail while not being in the field of view of the head mounted camera. The participants are not provided with the recipe beforehand. This setup simulates real-world scenarios where an assistant guides a user step by step through a recipe. To create a challenging benchmark, we intentionally include a broad range of realistic user errors. Prior to each recording session, participants are coached on common mistake types using practice recipes that differ from the one being filmed. We then run brief mock recording sessions, in which participants deliberately make mistakes and receive targeted feedback on the plausibility and naturalness of those mistakes. Finally, after each session, every video is manually inspected to ensure quality and realism.

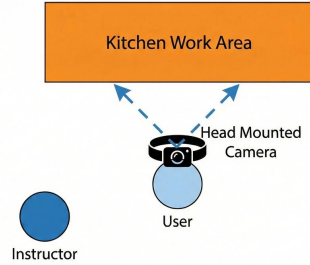


Figure 2: Recording setup: Dashed lines show the camera’s field of view.

Annotation and Verification. EGO-MC-BENCH features manually generated variants of everyday recipes, deliberately limiting the influence of prior knowledge so that evaluation emphasizes visual reasoning. The annotations in our EGO-MC-BENCH benchmark consist of instructions and feedbacks, provided by the instructor to the participant as described above. The voice recordings are transcribed and then manually verified by an independent annotator to ensure accuracy. Additionally, the annotators are tasked with classifying the transcripts into instructions or feedbacks.

Benchmark statistics. The benchmark contains ~ 10 hours of video data across 40 recording sessions. It features 7 participants in total and includes diverse kitchen setups (Fig. 1). It covers 559 recipe steps (across the 40 recording sessions) and 395 feedback messages corresponding to mistakes made by the participants (954 feedbacks in total). This also includes 22 clarification questions asked by the participants and subsequent clarifications provided by the instructor, along with 30 user comments usually acknowledging instructions or indicating preferences.

EGO-MC-BENCH includes two timing-based feedback types: *anticipatory* feedback, given when a mistake is imminent (e.g., warning a user not to add too much salt before they pour), and *post-error* feedback, given after a mistake has occurred when it could not have been anticipated (e.g., noticing that onions have already started to burn and advising the user to lower the heat). Irrecoverable mistakes appear only in the anticipatory setting, since once they occur, successful task completion is impossible. EGO-MC-BENCH contains an approximately equal number of mistakes for each type (49.2% vs 50.8%). This is very different from QICD [5] where all feedbacks are *post-error*. To further highlight the diversity of mistakes in the EGO-MC-BENCH benchmark, we additionally include an analysis of the types of mistakes, using the classification scheme from CaptainCook4D [30] in Fig. 3.

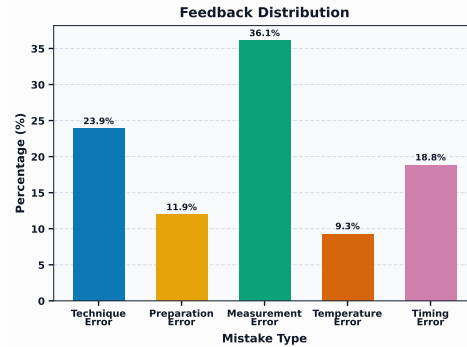


Figure 3: Distribution of feedbacks in EGO-MC-BENCH using classification of [30].

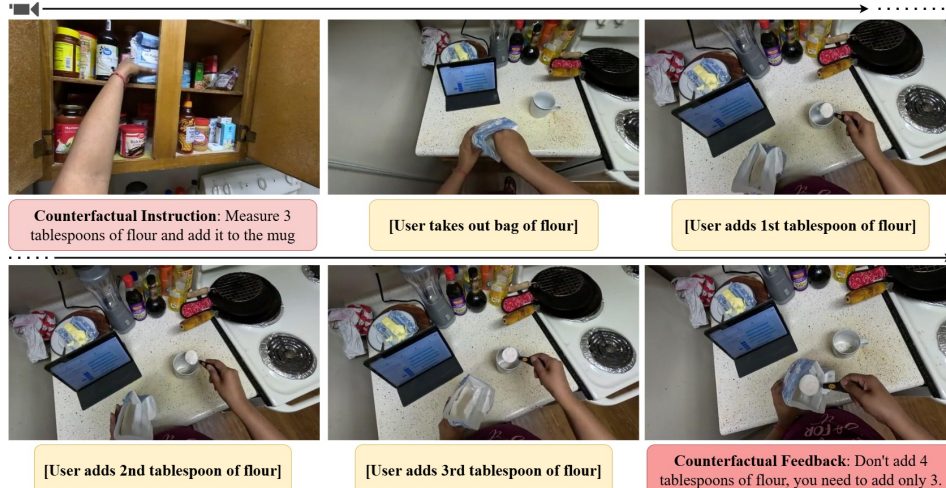


Figure 4: Counterfactual mistake annotation in EGO-COMIST.

Qualitative examples. We show qualitative examples from EGO-MC-BENCH in Fig. 1. In the top row, the assistant instructs the participant to add two teaspoons of salt; when only one is added, it intervenes with corrective feedback, after which the participant completes the step and receives confirmation. In the bottom row, the assistant instructs the participant to halve an apple and slice half of it thinly; it first corrects slice thickness and then the sliced quantity before the participant successfully completes the task. These examples illustrate the central challenge addressed by EGO-MC-BENCH: providing timely, appropriate feedback as soon as a mistake becomes apparent.

4 EGO-COMIST: Synthetic Counterfactual Mistakes

As EGO-MC-BENCH evaluates models in a realistic fully interactive setting, it serves as a gold-standard benchmark for studying training methods and datasets. Its stringent downstream evaluation also enables more exploratory data generation, including (noisy) synthetic data, since any drawback or benefit is directly reflected in task performance. Here, we introduce Qualcomm Interactive Cooking: EGO-COMIST, a counterfactual dataset of interactive instruction–feedback pairs. Built from existing non-interactive datasets [14, 20, 30], it approximates the EGO-MC-BENCH setting by providing feedback at the earliest point a mistake becomes apparent. Given a video clip of a recipe step, EGO-COMIST consists of two stages: (i) generating a counterfactual step with corresponding instruction and feedback, and (ii) inferring the timestamp at which the feedback should be delivered.

4.1 Stage 1: Counterfactual Instruction and Feedback Generation

In the first stage, we synthesize counterfactual variants of each recipe step by perturbing specific attributes: ingredient quantity (measurement errors), preparation method (preparation errors), cooking technique (technique errors), duration (timing errors), and temperature/heat setting (temperature errors). Together, these categories capture the vast majority of common cooking mistakes [30]. To this end, we extract the relevant attributes from the recipe step, i.e., quantity, cooking technique, preparation method, cooking time and cooking temperature, using a state of the art LLM (Gemini-2.5-Pro [38]). We then generate the corresponding counterfactual attribute, e.g., a counterfactual measurement, preparation method, cooking technique, heat setting or cooking time. Finally, given the original recipe step and the counterfactual attribute, we generate a counterfactual instruction and feedback pair. Next we describe this process in detail for the measurement error type and provide details of the remaining error types and prompts used in the appendix.

Measurement error. Given a recipe step that requires a specific quantity of an ingredient(s), e.g., *Measure 4 tablespoons of flour and add it to the mug*, we first ask the model to extract the specific quantity (i.e., *4 tablespoons*) and the corresponding ingredient (i.e., *flour*). Next, we ask the model to create a counterfactual quantity, that is reasonable for the given recipe step. In Fig. 4, the model chooses 3 tablespoons as the counterfactual quantity. Then, given the original quantity and

the counterfactual quantity, we ask the model to generate a new counterfactual instruction, i.e., *Measure 3 tablespoons of flour and add it to the mug*. This allows us now to construct the feedback corresponding to this counterfactual instruction: *Don't add 4 tablespoons of flour, you need to add only 3*. Additionally, to keep the dataset balanced, we generate an equal number of counterfactual instructions with smaller and larger measurements.

4.2 Stage 2: Counterfactual Feedback Timestamp Inference

At the second stage, we identify the appropriate timestamp for intervention and generate the corresponding (counterfactual) feedback. We first obtain step-by-step narrations of the input video clip. Given the original recipe step, the counterfactual recipe step, and the narration, we use a state-of-the-art LLM (Gemini-2.5-Pro) to infer when the feedback should be provided. This strategy is effective because the feedback timestamp is generated in an *oracle* setup, where the LLM has access to the counterfactual mistake and the step by step descriptions of the entire video clip, both of which are not available at inference time.

Step-by-step descriptions. To localize the intervention time, we require detailed, temporally aligned narrations focused on the user’s actions. When such descriptions are not already available in the dataset we generate them using a state-of-the-art video LLM (Qwen3-VL-32B-Instruct [13]) using a sliding-window approach that prompts the video LLM to describe overlapping video chunks (10 sec windows with 5 sec stride). We instruct the model to focus on hand-object interactions, which are critical for identifying errors, e.g., detecting when the user begins adding an extra tablespoon of flour. Given these narrations, we infer the appropriate feedback timestamp using a state-of-the-art LLM. We next describe this process for the measurement error type and provide details of the remaining error types in the appendix.

Measurement error. When the counterfactual measurement is smaller than the original quantity in the recipe step, we prompt the model to find the timestamped description at which it becomes clear that the person intends to use more than the counterfactual amount. For example, in Fig. 4, we ask the model to identify when the person is about to add a fourth tablespoon of flour, and use that timestamp for counterfactual feedback. When the counterfactual quantity is larger than the original quantity, we instead wait until it becomes clear that the person does not intend to add any more of the ingredient.

4.3 EGO-COMIST: Statistics and Human Evaluation

Using the pipeline described above we generate counterfactual mistakes for the following existing cooking datasets: CaptainCook4D [30], Ego4D [19] and Ego-Exo4D [20]. As CaptainCook4D already contains mistakes we use the recipe steps in CaptainCook4D without mistakes in our EGO-COMIST pipeline. In case of Ego4D and Ego-Exo4D, we leverage the Goal-Step [34] and key step annotations, respectively, to filter for appropriate videos containing cooking activities. We use these annotations to obtain video descriptions and leverage them to generate instructions. However, these descriptions are less detailed compared to CaptainCook4D and do not usually mention details like cooking temperature, measurement or duration. We therefore only generate preparation and technique errors from these datasets. Overall, EGO-COMIST contains 4969, 13847, 6271 instruction-feedback pairs from CaptainCook4D, Ego4D and Ego-Exo4D respectively.

Finally, we perform a user study of the quality of the annotations in EGO-COMIST. We randomly select 500 samples and first ask users to check if the example is valid. An example is invalid if the counterfactual instruction: 1. is not semantically different from the action in the clip, or, 2. is not feasible with the current ingredients, or, 3. does not belong to the annotated mistake type. The users found 87.1% of instruction-feedback pairs to be valid. Next, we asked them to find the appropriate time to provide the feedback, using the same criterion as for EGO-MC-BENCH. Fig. 5 shows the resulting distribution of errors between the annotated feedback timestamp and the timestamp

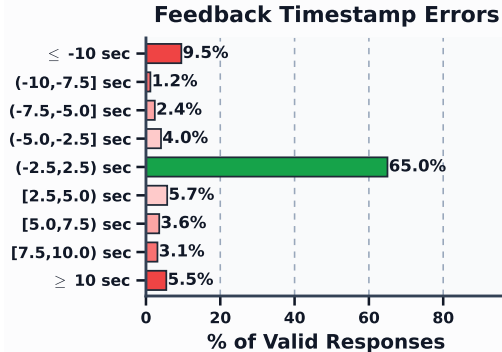


Figure 5: EGO-COMIST: Analysis of errors in timestamp inference (stage 2).

Table 2: Evaluation on the EGO-MC-BENCH, using the regular interval prompting strategy of [5].

Method	Instruction			Mistake		
	IC-Acc $\uparrow$	Prec. $\uparrow$	Rec. $\uparrow$	F1 $\uparrow$	BERT $\uparrow$	ROUGE-L $\uparrow$
Per-recipe step						
InternVL3.5-38B [40]	3.9	0.00	0.00	0.00	0.000	0.000
Qwen2.5-VL-32B [15]	27.3	0.00	0.00	0.00	0.000	0.000
Qwen3-VL-8B [13]	30.7	0.00	0.00	0.00	0.000	0.000
VideoLLaMA3-7B [49]	31.8	0.00	0.00	0.00	0.000	0.000
Qwen3.5-2B [31]	0.0	0.02	0.29	0.04	0.184	0.130
Qwen3.5-9B [31]	6.1	0.07	0.33	0.11	0.201	0.137
Qwen3.5-27B [31]	45.5	0.12	0.17	0.14	0.206	0.137
Qwen3-VL-32B [13]	6.8	0.10	0.34	0.16	0.068	0.092
Videollm-online [6]	2.7	0.02	0.38	0.05	0.265	0.201
LiveCC [7]	1.6	0.03	0.43	0.06	0.248	0.196
Gemini-2.5-Flash [36]	24.6	0.17	0.20	0.18	0.180	0.135
Gemini-3-Flash [9]	32.7	0.18	0.18	0.18	0.126	0.102
Full recipes						
Qwen3.5-27B [31]	30.3	0.05	0.13	0.07	0.201	0.136
Qwen3-VL-32B [13]	6.8	0.04	0.28	0.07	0.061	0.090
Gemini-3-Flash [9]	10.6	0.05	0.20	0.08	0.097	0.091

provided by the participants. We see that most predictions ($\sim 65\%$) are near-correct with error within $(-2.5, 2.5)$ sec. Extreme deviations (≥ 10 sec) remain a minority. Both “on time” annotations and annotations close to being on time (< 10 sec) provide useful training signals.

5 Experiments

In this section, we first evaluate state of the art video LLMs on our EGO-MC-BENCH benchmark and highlight challenges in live intervention in video streams. Subsequently, we finetune on synthetic counterfactual data from EGO-COMIST and show that it improves performance on EGO-MC-BENCH.

Evaluation metrics. We metrics based on QICD [5]: instruction completion accuracy (IC-Acc), mistake intervention (precision, recall, F1), and fluency (BERTScore, ROUGE-L). IC-Acc measures whether the model correctly identifies completed instructions; mistake intervention measures temporal alignment with ground-truth feedback; and fluency measures similarity to the reference feedback. Fluency is only directly comparable across models with similar mistake-intervention performance.

Evaluation protocol. We consider two evaluation protocols: (i) *per-recipe step*, (ii) *full recipe step* [5]. In the *per-recipe step* evaluation setup, the model is provided with an instruction corresponding to a single recipe step, e.g., *Take a large pan and add four tablespoons of olive oil.* as shown in Fig. 6, and the corresponding streaming video starting at the point where the instruction was provided in the groundtruth data. The task now is to provide appropriate feedbacks until the person successfully completes the recipe step. For the next recipe step, the input streaming video is re-started from the groundtruth recipe step start timestamp. In the *full recipe* protocol, the video is streamed continuously. However, this setup assumes that the user would move on to the next recipe step even when the model fails to generate a success confirmation message for the current step, which is unlikely in a real-world reactive setting (unlike the non-reactive setup of the Qualcomm Interactive Cooking Dataset [5]).

Evaluation of “turn-based” models. We begin by evaluating state of the art “turn-based” video LLMs on our EGO-MC-BENCH (Tab. 2): VideoLLaMA3-7B [49], Qwen2.5-VL-Instruct [15], Qwen3-VL-Instruct [13], Qwen3.5 [31] and Gemini-Flash [9, 38]. These are not streaming models and they only respond when prompted. We therefore adopt the regular-interval prompting protocol [5] for streaming evaluation, querying step completion and mistake intervention at fixed time intervals (a similar prompting strategy [33] has recently shown state of the art performance on streaming QA benchmarks [23, 24]). We also consider the narration models: Videollm-online [6] and LiveCC [7].

Table 3: Evaluation on our EGO-MC-BENCH. For the Qwen3-VL/3.5 models, we specify the dataset used for fine-tuning in brackets.

Method	Instruction			Mistake		
	IC-Acc $\uparrow$	Prec. $\uparrow$	Rec. $\uparrow$	F1 $\uparrow$	BERT $\uparrow$	ROUGE-L $\uparrow$
Per-recipe step						
ProAssist [51]	3.0	0.31	0.09	0.14	0.281	0.173
Qwen3.5-2B (QICD)	28.9	0.75	0.05	0.10	0.359	0.218
Qwen3.5-2B (EGO-CoMIST)	36.1	0.34	0.11	0.12	0.359	0.229
Qwen3-VL-2B (EGO-CoMIST+)	30.4	0.40	0.10	0.16	0.335	0.219
Qwen3.5-0.8B (EGO-CoMIST+)	30.5	0.29	0.03	0.06	0.339	0.183
Qwen3.5-2B (EGO-CoMIST+)	37.1	0.39	0.14	0.20	0.444	0.272
Full recipes						
ProAssist [51]	0.2	0.00	0.00	0.00	0.000	0.000
Qwen3.5-2B (QICD)	8.2	0.18	0.04	0.06	0.347	0.161
Qwen3.5-2B (EGO-CoMIST+)	19.7	0.30	0.08	0.13	0.433	0.278

We convert their generated narrations into interactive feedback by passing them to a helper LLM (Qwen3-32B [12]) that produces timely intervention messages (see appendix).

The results in Tab. 2, highlight that our EGO-MC-BENCH is highly challenging even for state of the art proprietary video LLMs such as Gemini-3-Flash. The mistake intervention F1 score remains low at 0.18 in the per-recipe step setting and importantly, the provided feedbacks are of limited utility as they do not match the ground truth feedbacks closely as evidenced by the BERT and ROUGE-L metrics. This is further highlighted in Fig. 6, which shows the feedbacks provided by Gemini-3-Flash given the instruction: *Take a large pan and add four tablespoons of olive oil*. The first feedback by Gemini-3-Flash asks the user to use a tablespoon to measure not a measuring cup, but the person has just taken out a tablespoon to measure. Afterwards, when the person sets the bottle of olive oil aside after adding just a single tablespoon of olive oil, Gemini-3-Flash assumes that the person has already completed the provided instruction. Of the open-weights models in Tab. 2, the Qwen3.5 series of models performs best especially in the mistake intervention and fluency metrics. However, some models such as InternVL3.5-38B, Qwen2.5-VL-32B, Qwen3-VL-8B-Instruct, VideoLLaMA3-7B show poor mistake intervention performance. This is because they are unable to follow the mistake intervention instruction, likely because these models are designed for “turn-based” question-answering tasks (see appendix). The streaming Videollm-online and LiveCC models perform poorly as they do not produce informative narrations at the right time to be useful for intervention mistakes. Finally, the performance in the full-recipe step setting is even weaker overall, with Gemini-3-Flash achieving the best mistake intervention performance and Qwen3.5-27B performing the best in the IC-Acc metric.

Evaluation of “streaming” video LLMs. Next, we evaluate streaming models that can provide interactive responses. To this end, we use our EGO-CoMIST dataset to fine-tune the Qwen3.5-2B, Qwen3.5-0.8B and Qwen3-VL-2B-Instruct [13] models. To enable interactive responses using the “turn-based” Qwen models, we attach an action head on the last transformer block of these models. The action head predicts a binary speak/stay silent action after every input frame (i.e., after the `<|vision_end|>` token) [5, 6, 29]. We also compare to the state of the art ProAssist [51] model which is trained on conversational data in the cooking domain.

We report the baselines in Tab. 3. The ProAssist model struggles to identify successful completion of instructions and provide fluent feedbacks. The Qwen3.5-2B model fine-tuned on our EGO-CoMIST dataset performs better on all metrics. To highlight the effectiveness of our EGO-CoMIST dataset, we consider an ablation where the Qwen3.5-2B model is finetuned instead on the QICD [5] dataset. We see significant degradation of mistake intervention performance. This is to be expected as in QICD feedbacks are provided post-error and the participants are non-reactive. Interestingly, we find that mixing QICD together with our EGO-CoMIST dataset (EGO-CoMIST+ in Tab. 3) improves performance. This occurs only when we explicitly specify the qualitative difference in intervention strategies in the model prompt, leading to a multi-task learning setup. The Qwen3.5-2B and Qwen3-VL-2B models trained on EGO-CoMIST+ outperform both ProAssist and Qwen3.5-2B (trained only

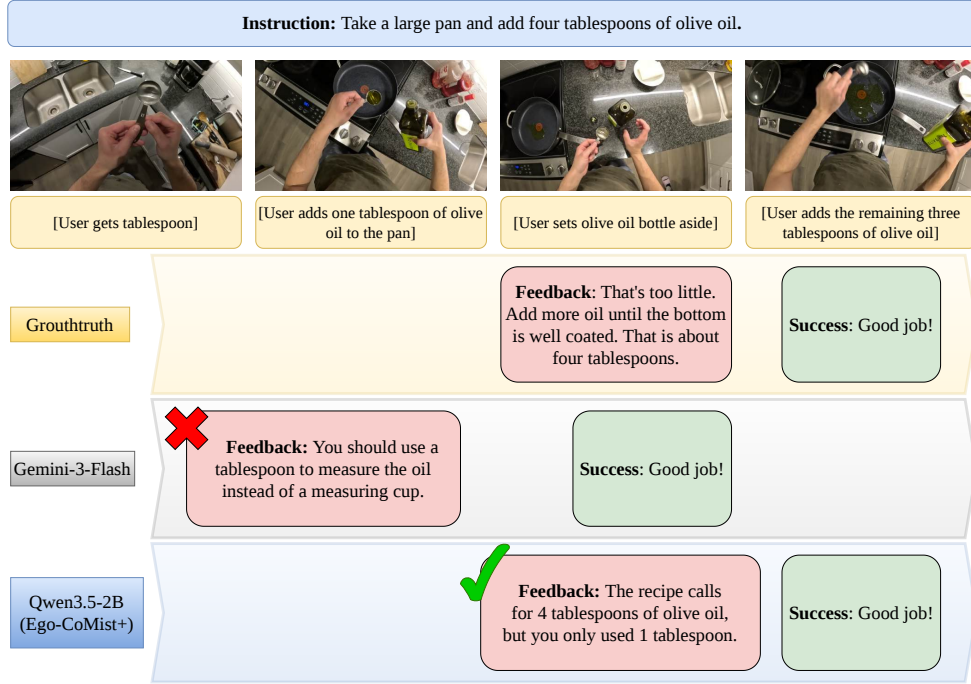


Figure 6: EGO-MC-BENCH streaming interventions: Gemini-3-Flash [9] produces incorrect feedbacks and is unable to intervene when the person adds only one tablespoon of olive oil. The Qwen3.5-2B model finetuned on EGO-COMIST+ intervenes at the appropriate time.

on QICD), again highlighting the effectiveness of our EGO-COMIST dataset. Note that, the Qwen3.5-2B without fine-tuning on EGO-COMIST+ performs very poorly as shown in Tab. 2. Finetuning on EGO-COMIST+ significantly improves its IC-Acc from 0.0 to 33.0 and mistake F1 from 0.04 to 0.18 and enables the model to react interactively to the input video stream – matching performance of the proprietary Gemini-3-Flash model.

In the *full recipe* evaluation setup, the performance of ProAssist is very weak: this is because as in the per-recipe step scenario, the model is unable to predict successful completion of recipe steps in the reactive setup of EGO-MC-BENCH. Moreover, unlike in the per-recipe step setup in the full-recipe setup errors accumulate over recipe steps, leading to poor performance. Finally, we again see that training on our EGO-COMIST dataset leads to significant improvement over training just on QICD.

6 Conclusion

We introduced Qualcomm Interactive Cooking: EGO-MC-BENCH, a benchmark that addresses a key gap in interactive step-by-step task guidance: the evaluation of video LLMs in *reactive* assistant–user interactions. We further proposed Qualcomm Interactive Cooking: EGO-COMIST, a counterfactual synthetic dataset by converting non-interactive cooking clips into supervised instances of proactive intervention, providing both counterfactual instructions and precisely timed feedback signals. Our evaluation shows that EGO-MC-BENCH is challenging for state-of-the-art MLLMs: off-the-shelf models struggle to detect mistakes and to time interventions correctly. Fine-tuning with EGO-COMIST consistently improves mistake detection, intervention timing, and instruction-completion accuracy.

Limitations and Broader Impacts. Our work is focused on the cooking domain through EGO-MC-BENCH and EGO-COMIST. While we show that fine-tuning models on EGO-COMIST leads to improved performance, these models are still far away from real-world deployment. Also, video LLMs can produce harmful and biased content, make incorrect claims and produce wrongful advice. This needs to be taken into account when interacting with, deploying or building on these models. It also has to be taken into account that any computer vision model processing visual information about human activities could in principle extract information beyond what is required for the use-case.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob Menick, Sebastian Borgeaud, Andrew Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karen Simonyan. Flamingo: a visual language model for few-shot learning. In *NeurIPS*, 2022.
- [3] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A versatile vision-language model for understanding, localization. *Text Reading, and Beyond*, 2, 2023.
- [4] Yuwei Bao, Keunwoo Peter Yu, Yichi Zhang, Shane Storks, Itamar Bar-Yossef, Alexander De La Iglesia, Megan Su, Xiao-Lin Zheng, and Joyce Chai. Can foundation models watch, talk and guide you step by step to make a cake? In *EMNLP Findings*, 2023.
- [5] Apratim Bhattacharyya, Bicheng Xu, Sanjay Haresh, Reza Pourreza, Litian Liu, Sunny Panchal, Pulkit Madan, Leonid Sigal, and Roland Memisevic. Can multi-modal llms provide live step-by-step task guidance? In *NeurIPS*, 2025.
- [6] Joya Chen, Zhaoyang Lv, Shiwei Wu, Kevin Qinghong Lin, Chenan Song, Difei Gao, Jia-Wei Liu, Ziteng Gao, Dongxing Mao, and Mike Zheng Shou. Videollm-online: Online video large language model for streaming video. In *CVPR*, 2024.
- [7] Joya Chen, Ziyun Zeng, Yiqi Lin, Wei Li, Zejun Ma, and Mike Zheng Shou. Livecc: Learning video llm with streaming speech transcription at scale. In *CVPR*, 2025.
- [8] Dima Damen, Hazel Doughty, Giovanni Maria Farinella, Antonino Furnari, Jian Ma, Evangelos Kazakos, Davide Moltisanti, Jonathan Munro, Toby Perrett, Will Price, and Michael Wray. Rescaling egocentric vision: Collection, pipeline and challenges for epic-kitchens-100. In *IJCV*, 2022.
- [9] Google Deepmind. “Gemini 3 Flash.”. <https://deepmind.google/models/gemini/flash/>, 2025. [Online; accessed May-2026].
- [10] Shangzhe Di, Zhelun Yu, Guanghao Zhang, Haoyuan Li, Hao Cheng, Bolin Li, Wanggui He, Fangxun Shu, Hao Jiang, et al. Streaming video question-answering with in-context video kv-cache retrieval. In *ICLR*, 2025.
- [11] Xin Ding, Hao Wu, Yifan Yang, Shiqi Jiang, Donglin Bai, Zhibo Chen, and Ting Cao. Stream-mind: Unlocking full frame rate streaming video dialogue through event-gated cognition. *CoRR*, abs/2503.06220, 2025.
- [12] An Yang et al. Qwen3 technical report. *CoRR*, abs/2505.09388, 2025.
- [13] Bai et. al. Qwen3-vl technical report. *CoRR*, abs/2511.21631, 2025.
- [14] Grauman et. al. Ego4d: Around the world in 3,000 hours of egocentric video. In *CVPR*, 2022.
- [15] Shuai Bai et. al. Qwen2.5-vl technical report. *CoRR*, abs/2502.13923, 2025.
- [16] Chaoyou Fu, Yuhang Dai, Yongdong Luo, Lei Li, Shuhuai Ren, Renrui Zhang, Zihan Wang, Chenyu Zhou, Yunhang Shen, Mengdan Zhang, et al. Video-mme: The first-ever comprehensive evaluation benchmark of multi-modal llms in video analysis. In *CVPR*, 2025.
- [17] Shenghao Fu, Qize Yang, Yuan-Ming Li, Yi-Xing Peng, Kun-Yu Lin, Xihan Wei, Jian-Fang Hu, Xiaohua Xie, and Wei-Shi Zheng. Vispeak: Visual instruction feedback in streaming videos. In *ICCV*, 2025.

- [18] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *CoRR*, abs/2407.21783, 2024.
- [19] Kristen Grauman, Andrew Westbury, Eugene Byrne, Zachary Chavis, Antonino Furnari, Rohit Girdhar, Jackson Hamburger, Hao Jiang, Miao Liu, Xingyu Liu, Miguel Martin, Tushar Nagarajan, Ilija Radosavovic, Santhosh Kumar Ramakrishnan, Fiona Ryan, Jayant Sharma, Michael Wray, Mengmeng Xu, Eric Zhongcong Xu, Chen Zhao, Siddhant Bansal, et al. Ego4d: Around the world in 3, 000 hours of egocentric video. In *CVPR*, 2022.
- [20] Kristen Grauman, Andrew Westbury, Lorenzo Torresani, Kris Kitani, Jitendra Malik, Triantafyllos Afouras, Kumar Ashutosh, Vijay Baiyya, Siddhant Bansal, Bikram Boote, et al. Ego-exo4d: Understanding skilled human activity from first- and third-person perspectives. *CoRR*, abs/2311.18259, 2023.
- [21] Kairui Hu, Penghao Wu, Fanyi Pu, Wang Xiao, Yuanhan Zhang, Xiang Yue, Bo Li, and Ziwei Liu. Video-mmmu: Evaluating knowledge acquisition from multi-discipline professional videos. *CoRR*, abs/2501.13826, 2025.
- [22] Wei Li, Bing Hu, Rui Shao, Leyang Shen, and Liqiang Nie. LION-FS: fast & slow video-language thinker as online video assistant. In *CVPR*, 2025.
- [23] Yifei Li, Junbo Niu, Ziyang Miao, Chunjiang Ge, Yuanhang Zhou, Qihao He, Xiaoyi Dong, Haodong Duan, Shuangrui Ding, Rui Qian, et al. Ovo-bench: How far is your video-llms from real-world online video understanding? *CoRR*, abs/2501.05510, 2025.
- [24] Junming Lin, Zheng Fang, Chi Chen, Zihao Wan, Fuwen Luo, Peng Li, Yang Liu, and Maosong Sun. Streamingbench: Assessing the gap for mllms to achieve streaming video understanding. *CoRR*, abs/2411.03628, 2024.
- [25] Antoine Miech, Dimitri Zhukov, Jean-Baptiste Alayrac, Makarand Tapaswi, Ivan Laptev, and Josef Sivic. Howto100m: Learning a text-video embedding by watching hundred million narrated video clips. In *ICCV*, 2019.
- [26] Zhenyu Ning, Guangda Liu, Qihao Jin, Wenchao Ding, Minyi Guo, and Jieru Zhao. Livevlm: Efficient online video understanding via streaming-oriented KV cache and retrieval. *CoRR*, abs/2505.15269, 2025.
- [27] OpenAI. “Introducing GPT-5.2.”. <https://openai.com/index/introducing-gpt-5-2/>, 2025. [Online; accessed March-2025].
- [28] Sunny Panchal, Apratim Bhattacharyya, Guillaume Berger, Antoine Mercier, Cornelius Böhm, Florian Dietrichkeit, Reza Pourreza, Xuanlin Li, Pulkit Madan, Mingu Lee, Mark Todorovich, Ingo Bax, and Roland Memisevic. What to say and when to say it: Live fitness coaching as a testbed for situated interaction. In *NeurIPS*, 2024.
- [29] Sunny Panchal, Apratim Bhattacharyya, Guillaume Berger, Antoine Mercier, Cornelius Bohm, Florian Dietrichkeit, Reza Pourreza, Xuanlin Li, Pulkit Madan, Mingu Lee, Mark Todorovich, Ingo Bax, and Roland Memisevic. What to say and when to say it: Live fitness coaching as a testbed for situated interaction. In *NeurIPS*, 2024.
- [30] Rohith Peddi, Shivvrat Arya, Bharath Challa, Likhitha Pallapothula, Akshay Vyas, Bhavya Gouripeddi, Qifan Zhang, Jikai Wang, Vasundhara Komaragiri, Eric D. Ragan, Nicholas Ruozzi, Yu Xiang, and Vibhav Gogate. Captaincook4d: A dataset for understanding errors in procedural activities. In *NeurIPS*, 2024.
- [31] QwenTeam. “Qwen3.5: Towards Native Multimodal Agents.”. <https://qwen.ai/blog?id=qwen3.5>, 2026. [Online; accessed May-2026].
- [32] Fadime Sener, Dibyadip Chatterjee, Daniel Sheleпов, Kun He, Dipika Singhania, Robert Wang, and Angela Yao. Assembly101: A large-scale multi-view video dataset for understanding procedural activities. In *CVPR*, 2022.

- [33] Yujiao Shen, Shulin Tian, Jingkang Yang, and Ziwei Liu. A simple baseline for streaming video understanding. *CoRR*, abs/2604.02317, 2026.
- [34] Yale Song, Eugene Byrne, Tushar Nagarajan, Huiyu Wang, Miguel Martin, and Lorenzo Torresani. Ego4d goal-step: Toward hierarchical understanding of procedural activities. In *NeurIPS*, 2023.
- [35] Yansong Tang, Dajun Ding, Yongming Rao, Yu Zheng, Danyang Zhang, Lili Zhao, Jiwen Lu, and Jie Zhou. COIN: A large-scale dataset for comprehensive instructional video analysis. In *CVPR*, 2019.
- [36] Gemini Team. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *CoRR*, abs/2507.06261, 2025.
- [37] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: A family of highly capable multimodal models. *CoRR*, abs/2312.11805, 2023.
- [38] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *CoRR*, abs/2403.05530, 2024.
- [39] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *CoRR*, abs/2409.12191, 2024.
- [40] Weiyun Wang, Zhangwei Gao, Lixin Gu, Hengjun Pu, Long Cui, Xingguang Wei, Zhaoyang Liu, Linglin Jing, Shenglong Ye, Jie Shao, et al. Internvl3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency. *CoRR*, abs/2508.18265, 2025.
- [41] Xin Wang, Taein Kwon, Mahdi Rad, Bowen Pan, Ishani Chakraborty, Sean Andrist, Dan Bohus, Ashley Feniello, Bugra Tekin, Felipe Vieira Frujeri, Neel Joshi, and Marc Pollefeys. Holoassist: an egocentric human interaction dataset for interactive AI assistants in the real world. In *ICCV*, 2023.
- [42] Yuxuan Wang, Yueqian Wang, Bo Chen, Tong Wu, Dongyan Zhao, and Zilong Zheng. Omnimmi: A comprehensive multi-modal interaction benchmark in streaming video contexts, 2025.
- [43] Zeqing Wang, Wentao Wan, Qiqing Lao, Runmeng Chen, Minjie Lang, Xiao Wang, Keze Wang, and Liang Lin. Towards top-down reasoning: An explainable multi-agent approach for visual question answering. *CoRR*, abs/2311.17331, 2025.
- [44] Haoning Wu, Dongxu Li, Bei Chen, and Junnan Li. Longvideobench: A benchmark for long-context interleaved video-language understanding. In *NeurIPS*, 2024.
- [45] Jiaer Xia, Peixian Chen, Mengdan Zhang, Xing Sun, and Kaiyang Zhou. Streaming video instruction tuning. *CoRR*, abs/2512.21334, 2025.
- [46] Jin Xu, Zhifang Guo, Hangrui Hu, Yunfei Chu, Xiong Wang, Jinzheng He, Yuxuan Wang, Xian Shi, Ting He, Xinfu Zhu, et al. Qwen3-omni technical report. *CoRR*, abs/2501.13826, 2025.
- [47] Ruyi Xu, Guangxuan Xiao, Yukang Chen, Liuning He, Kelly Peng, Yao Lu, and Song Han. Streamingvlm: Real-time understanding for infinite video streams. *CoRR*, abs/2510.09608, 2025.
- [48] Zhenyu Yang, Yuhang Hu, Zemin Du, Dizhan Xue, Shengsheng Qian, Jiahong Wu, Fan Yang, Weiming Dong, and Changsheng Xu. Svbench: A benchmark with temporal multi-turn dialogues for streaming video understanding. *CoRR*, abs/2502.10810, 2025.
- [49] Boqiang Zhang, Kehan Li, Zesen Cheng, Zhiqiang Hu, Yuqian Yuan, Guanzheng Chen, Sicong Leng, Yuming Jiang, Hang Zhang, Xin Li, Peng Jin, Wenqi Zhang, Fan Wang, Lidong Bing, and Deli Zhao. Videollama 3: Frontier multimodal foundation models for image and video understanding, 2025.

- [50] Haoji Zhang, Yiqin Wang, Yansong Tang, Yong Liu, Jiashi Feng, Jifeng Dai, and Xiaojie Jin. Flash-vstream: Memory-based real-time understanding for long video streams. *CoRR*, abs/2406.08085, 2025.
- [51] Yichi Zhang, Xin Luna Dong, Zhaojiang Lin, Andrea Madotto, Anuj Kumar, Babak Damavandi, Joyce Chai, and Seungwhan Moon. Proactive assistant dialogue generation from streaming egocentric videos. In *EMNLP*, 2025.
- [52] Luowei Zhou, Chenliang Xu, and Jason Corso. Towards automatic learning of procedures from web instructional videos. In *AAAI*, 2018.

A Appendix

Here we provide: 1. Additional examples from our EGO-MC-BENCH benchmark. 2. Additional qualitative examples from state of the art models on our EGO-MC-BENCH benchmark. 3. Additional details of the evaluation of “turn-based” video LLMs. 4. Additional details of the evaluation of streaming narration models. 5. Additional details of the evaluation metrics. 6. Additional training details. 7. Additional details of the EGO-COMIST synthetic data generation pipeline. 8. Prompts used at both stages of the EGO-COMIST synthetic data generation process.

B EGO-MC-BENCH: Additional Examples

We provide additional qualitative examples from our EGO-MC-BENCH benchmark in Fig. 7.

In the top row, the assistant first provides the instruction: *Now, grab 2 tomatoes and dice them into small cubes.* The user then starts to dice the tomatoes, but make large cubes. So, the assistant intervenes and provides the feedback: *You need to dice the tomatoes in smaller cubes.* The user then dices the tomatoes correctly. But, instead of adding just two tomatoes, the user starts to dice a third tomato. The assistant intervenes again and provides the feedback: *You need to add only two tomatoes.* The user then puts the tomato back and the assistant acknowledges the successful completion of the recipe step.

In the bottom row, the assistant first provides the instruction: *Now cover each slice of bread with shredded mozzarella cheese.* The user then starts to add shredded mozzarella cheese to the slices of bread. But the user stops after adding too little cheese. So, the assistant intervenes and provides the feedback: *That’s too little. You might want to add a little bit more to cover each slice.* The user then adds way too much cheese on one of the slices. The assistant intervenes again and provides the feedback: *Oh, that’s too much. You might need to take out some of the mozzarella from the toast that has too much cheese.* The user then re-distributes the cheese correctly and the assistant acknowledges the successful completion of the recipe step.

These examples again highlight the core challenge of our EGO-MC-BENCH benchmark: intervention by providing appropriate feedback as soon as a mistake becomes apparent, guiding the user towards successful completion.

C EGO-MC-BENCH: Additional Qualitative Examples

In addition to Fig. 6 in the main paper, we show a qualitative example in Fig. 8 to highlight the weak performance of state of the art video LLMs, e.g., Gemini-3-Flash [9]. The person is trying to execute the instruction *Take a bowl and put 3 tablespoons of sesame oil in it* in Fig. 8. The first feedback by Gemini-3-Flash asks the person to roll up their sleeves before starting to cook although this is irrelevant for this recipe step. Next, Gemini-3-Flash asks the person to use a measuring spoon even though the person is already using a measuring spoon. The Qwen3.5-VL model trained on our EGO-COMIST+ dataset is able to correctly provide the feedback that the person should add three tablespoons of sesame oil and not stop at two. Thus, successfully guiding the person to completion of the recipe step.

D Additional Details: Evaluation of “Turn-Based” Models

As mentioned in the main paper, we use the regular interval prompting strategy [5] to evaluate state of the art “turn-based” video LLMs which cannot respond interactively to video input streams: VideoLLaMA3-7B [49], InternVL3.5-38B [40], Qwen2.5-VL-Instruct [15] series, Qwen3-VL-Instruct [13] series, Qwen3.5 [31] series, and Gemini-Flash [38, 9]. We use a time-interval of 5 seconds [5] to balance accuracy and inference speed. At every turn we first prompt the video LLM to check if the person has completed the current recipe step:

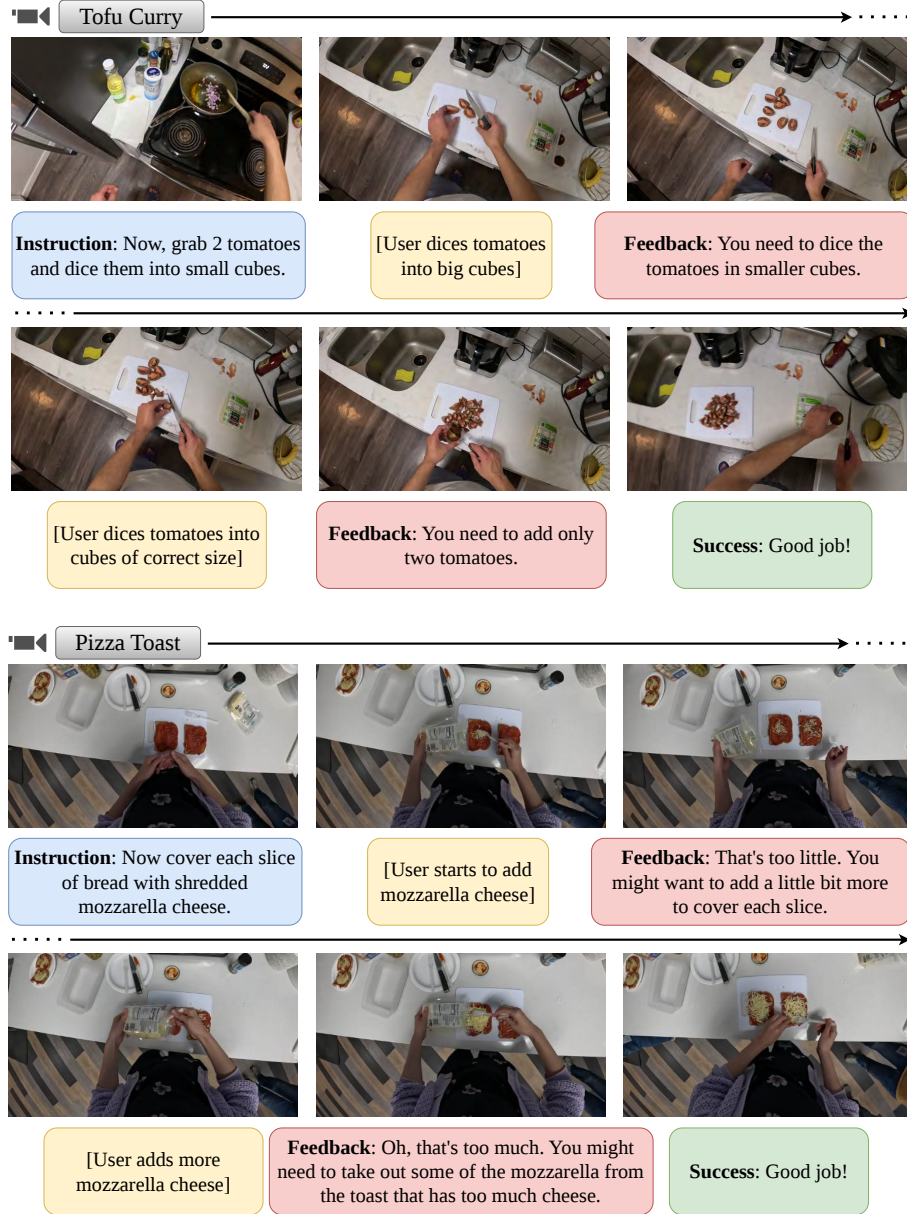


Figure 7: Our EGO-MC-BENCH benchmark includes interventions with appropriate feedback whenever a mistake is detected, guiding the user towards successful goal completion across recipe steps.

Check if Recipe Step Complete

You are an expert cooking assistant who is observing a person cook.

##INSTRUCTIONS:

The person is currently at the following recipe step: [recipe_step]. Has the person already completed the recipe step? If the person has completed the recipe step answer 'YES' else answer 'NO'. If you answer 'YES' describe why you think the person already completed the recipe step.

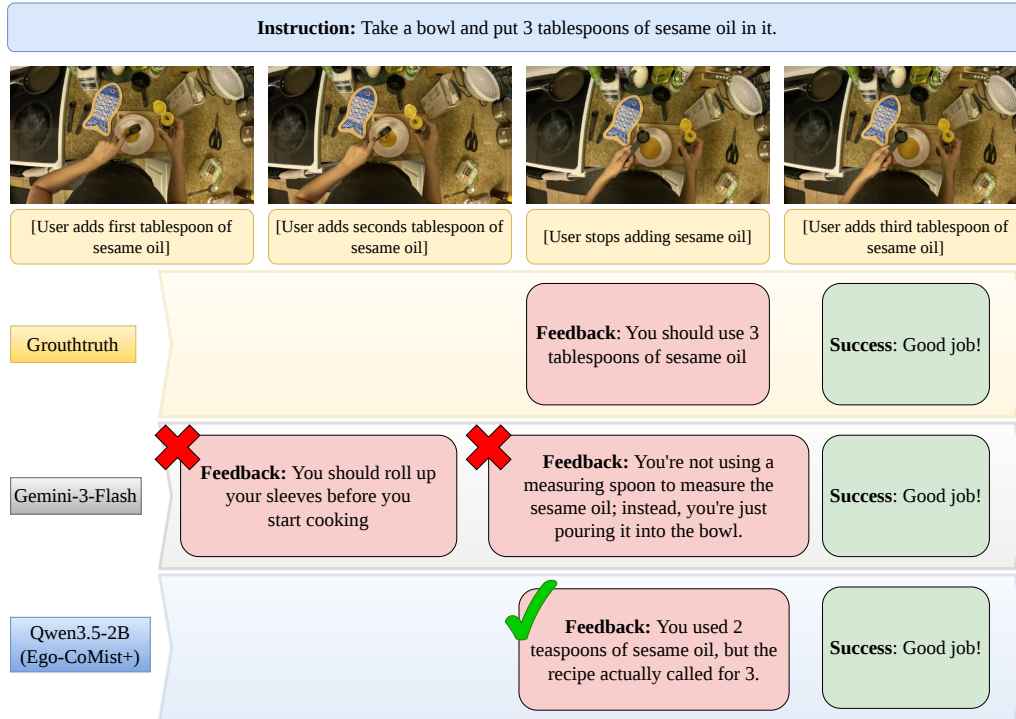


Figure 8: Additional EGO-MC-BENCH streaming interventions: Gemini-3-Flash [9] produces incorrect feedbacks and is unable to intervene when the person adds only two tablespoons of sesame oil. The Qwen3.5-2B model trained on our EGO-COMIST+ dataset intervenes at the appropriate time.

We found that asking the model to articulate why the recipe step is complete led to a boost in performance across models. Furthermore, we did not find any significant change in performance for alternate wordings of the prompt across models.

Now, if the model answers that the recipe step is not yet complete, we next ask the model to check if the person has made a mistake. For Qwen3-VL-Instruct [13], Qwen3.5 [31] and Gemini-Flash [38, 9] series, we use the following prompt which describes all mistake types in detail and also describes how to detect and intervene these mistakes. Also, in order to make sure that the video LLM does not repeat feedback for the same mistake again and again (as the past video frames are available in the context window), we store recently generated feedbacks (of the last 1 minute) in an array and ask the video LLM not to repeat the same feedback. Note that, this prompt was designed iteratively, where we added details until we saw no further improvements in performance.

Check for Mistake
You are an expert cooking assistant who is observing a person cook. You should look out for mistakes made by the person. ##INSTRUCTIONS: The person is trying to complete the following recipe step: [recipe_step]. Your task is to check if the person is about to make or has already made a mistake. Mistakes occur when the person performs actions that deviates from the instruction and DIRECTLY INTERFERES WITH SUCCESSFUL INSTRUCTION COMPLETION. Do not penalize actions that do not directly interfere with instruction completion (e.g. washing broccoli before cutting, if the instruction just says ‘cut broccoli’).

Here are some common types of mistakes that you should look out for: 1. Technique Error: A mistake in how a step is physically performed. Examples include chopping with the wrong motion, stirring when folding is required, or spilling during transfer, producing uneven cuts or texture issues even when tools and amounts are right. Ignore minor technique errors such as not holding or gripping objects properly, holding with a risk of dropping etc. that do not interfere directly with recipe completion.

2. Preparation Error: A setup mistake before executing the step . Using the wrong or dirty utensil, not washing/peeling/draining ingredients, insufficient draining of fluid, cutting/chopping without peeling which makes correct execution difficult or unsafe.

3. Measurement Error: An error in quantity — wrong counts, volumes, weights, or units. Mixing up teaspoons and tablespoons, misreading a scale, or miscounting items leads to off ratios and predictable taste or texture problems.

4. Temperature Error: A mistake in heat level or thermal state — the applied temperature, starting temperature, or thermal transition is wrong. Not preheating, using the wrong microwave power, overheating oil, or adding cold liquid when warm is required often causes burning, undercooking, or split emulsions.

5. Timing Error: A mistake in duration – over- or under-doing a step or skipping required rests, proofs, or cooling periods. Overcooking, underblending, or cutting resting time short typically yields incorrect doneness or unstable textures.

Do not repeat previously detected mistakes. Here are the feedbacks corresponding to previously detected mistakes:[prev_detected_mistakes]. Assume the recipe step is still in progress. Your task is to identify any mistake that’s already visible in the partially completed step. Do not penalize partially completed recipe steps. If you observe a mistake answer ‘YES’, else ‘No’. Your response MUST BEGIN WITH ‘YES’ or ‘NO’. In case you answer ‘YES’, please follow with a concise feedback to the user describing the mistake (i.e. YES. <feedback>.). Directly address the person.

For the VideoLLaMA3-7B [49], InternVL3.5-38B [40], Qwen2.5-VL-Instruct [15] series, we used the following simple prompt:

Check for Mistake

You are an expert cooking assistant who is observing a person cook. You should look out for mistakes made by the person.

##INSTRUCTIONS:

The person is trying to complete the following recipe step: [recipe_step]. Your task is to check if the person is about to make or has already made a mistake. Mistakes occur when the person performs actions that deviates from the instruction and **DIRECTLY INTERFERES WITH SUCCESSFUL INSTRUCTION COMPLETION**. Assume the recipe step is still in progress. Your task is to identify any mistake that’s already visible in the partially completed step.

Do not penalize partially completed recipe steps. If you observe a mistake answer ‘YES’, else ‘No’. Your response MUST BEGIN WITH ‘YES’ or ‘NO’. In case you answer ‘YES’, please follow with a concise feedback to the user describing the mistake (i.e. YES. <feedback>.). Directly address the person.

As shown in Tab. 2, even with this simple prompt these models (almost) never predict that a mistake has occurred. Adding the additional details as in the prompt for the Qwen3-VL-Instruct [13], and Gemini-Flash [9, 38] series does not improve performance.

E Additional Details: Evaluation of Streaming Narration Models

As mentioned in the main paper, for streaming narration models such as Videollm-online [6] and LiveCC [7], we convert their generated narrations into interactive feedback by passing them to a helper LLM (Qwen3.5-27B [31]). Note that in case of LiveCC, it produces narrations after every 1 second of video. We store these narrations in a buffer ([narration_list]) and after every 5 seconds

(similar to the case of the “turn-based” video LLMs described above in Sec. D), we prompt the helper Qwen3.3-27B [31] LLM as follows:

Check if Recipe Step Complete (Streaming Narration Models)
You are an intelligent chatbot that is judging another system which narrates human cooking videos. Given a high level action instruction and a list of narrations generated from the system, your job is to decide if the narration is correct and shows completion of the instruction. ##INSTRUCTIONS: List of narrations: [narration_list] Answer ‘YES’ if the instruction is completed otherwise output ‘NO’.

If the helper LLM predicts that the instruction is not complete, we next ask the helper LLM to check if the person has made a mistake (similar to the case of the “turn-based” video LLMs). We use the simple prompt described in Sec. D used with the VideoLLaMA3-7B [49], InternVL3.5-38B [40], Qwen2.5-VL-Instruct [15] series, Qwen3.5 [31] series models. But instead of the video we provide the narrations from the streaming narration model as input. Adding additional details to the prompt as for the Qwen3-VL-Instruct/Qwen3.5, and Gemini-Flash series does not improve performance.

F Additional Details: Evaluation Metrics

To compute the IC-Acc, mistake intervention and mistake fluency metrics, we use a temporal window size of 30 seconds [5].

The mistake fluency scores are comparable only at the same mistake detection levels, because the scores are computed only for true positives. If the true positive rate is low, then the model could get higher scores by making a few very good predictions. But in practice, a model with a higher true positive rate is preferred.

G Additional Training Details

As described in the main paper, we fine-tune the language backbone of the Qwen3-VL-2B-Instruct, Qwen3.5-0.8B, Qwen3.5-2B models using LoRA (dim = 64) for $\sim 100k$ iterations (based on validation loss). We use the AdamW optimizer with a learning rate of 2×10^{-4} and a cosine annealing learning rate schedule for 100k iterations until a learning rate of 1×10^{-6} . We use a batch size of 32 using 8 Nvidia H100 GPUs (with 4 gradient accumulation steps). For all Qwen3-VL/3.5 models in Tab. 3 trained using EGO-CoMIST+ data, we additionally train them on action segments from Ego4D Goal-Step [34] to improve IC-Acc scores. We convert the leaf action descriptions in Ego4D Goal-Step into instructions (using Qwen3-8B) and add success confirmation messages at the end of the action. Overall composition of EGO-CoMIST+ is 60% EGO-CoMIST, 30% QICD, and 10% Ego4D Goal-Step actions.

H EGO-CoMIST: Data Generation Process

H.1 Stage 1: Counterfactual Instruction and Feedback Generation

Continuing from Sec. 4.1 in the main paper, we describe the stage 1 of the counterfactual data generation pipeline for the preparation, technique, temperature and timing errors.

Preparation error. Given a recipe step which uses a specific preparation method on an ingredient or item (object) used in the cooking process, we first ask the LLM to extract the preparation method and the object. For example, given the instruction: *Coat the 6 oz. ramekin cup with cooking spray*, the preparation method is *coat with cooking spray* and the object is the *ramekin cup*. Then, we ask the LLM to propose an alternative preparation method that for the object that aligns with the original recipe step. For example: *coat with cooking spray* can be replaced with *grease with butter*. Given the original preparation method and the alternative preparation method, we can now generate the

counterfactual instruction and feedback messages: *Grease the 6 oz. ramekin cup with butter* and *You should grease the ramekin cup with butter, not coat with cooking spray.*

Technique error. Similar to the counterfactual instruction and feedback generation process for preparation errors described above, given a recipe step, we extract the specific cooking technique used along with the ingredient or item (object) used in the specific technique. For example, given the instruction: *Cut the English muffin into two pieces with a knife*, the technique is *cut with a knife* and the object is the *English muffin*. Then, we ask the LLM to propose an alternative technique that aligns with the original recipe step. For example: *cut with a knife* can be replaced with *split with a fork*. Given, the original and the alternative technique, we can now generate the counterfactual instruction and feedback messages: *Split the English muffin into two pieces with a fork* and *Don't use a knife to cut the muffin, use a fork to split it instead.*

Temperature error. Given a recipe step that involves cooking at a specific temperature, heat setting or object at a thermal state (e.g., hot/cold), we ask the LLM to suggest an alternative reasonable temperature, heat setting or a thermal state. For example, given the recipe step *Microwave the plate, covered, on high for 1.5 minutes*, we can generate the counterfactual instruction: *Microwave the plate, covered, at medium heat for 1.5 minutes* and the feedback: *Be careful, this step calls for medium heat, not high, to avoid overcooking.*

Timing error. Given a recipe step that involves cooking for a specific duration, we ask the LLM to suggest an alternative reasonable duration. For example, given the recipe step *Microwave on high for 30 seconds*, we can generate the counterfactual instruction: *Microwave on high for 45 seconds* and the feedback: *You did not microwave for long enough. You should microwave on high for 45 seconds.* Additionally, to keep the dataset balanced, we generate an equal number of counterfactual instructions with shorter and longer durations.

H.2 Stage 2: Counterfactual Feedback Timestamp Inference

Continuing from Sec. 4.2 in the main paper, we describe the stage 2 of the counterfactual data generation pipeline for the preparation, technique, temperature and timing errors.

Preparation error. In case of preparation errors, we prompt the LLM to find the timestamp at which the person starts to use the incorrect preparation method. For example, if the original instruction was *Coat the 6 oz. ramekin cup with cooking spray* and the counterfactual instruction is *Grease the 6 oz. ramekin cup with butter*, we prompt the LLM to look for the description which states that the person is starting to coat the ramekin cup with cooking spray. We use the timestamp of this description as the counterfactual feedback timestamp.

Technique error. In case of technique errors, we prompt the LLM to find the timestamp where the person starts to use the incorrect technique. For example, if the original instruction was *Cut the English muffin into two pieces with a knife* and the counterfactual instruction is *Split the English muffin into two pieces with a fork*, we prompt the LLM to find the description from the step by step descriptions which states that the person starts to cut the english muffin into two pieces with a knife. We use the timestamp of this description as the counterfactual feedback timestamp.

Temperature error. In case of temperature errors, we prompt the LLM to find the timestamp where it becomes clear that the person has used the wrong heat setting. For example, if the original instruction calls for microwaving at high heat and the counterfactual instruction calls for microwaving at medium heat, we use the timestamp where the person sets the heat setting on the microwave.

Timing error. In case the counterfactual amount of time (y seconds) is smaller than the original amount of time (x seconds) in the recipe step, we provide the feedback at least y seconds from when the cooking process begins. For example, if the original recipe calls for microwaving for 45 seconds and the counterfactual amount of time is 30 seconds, we wait at least 30 seconds from the beginning of the microwaving process. In case the counterfactual amount of time is larger than the original amount of time, we provide the feedback after the cooking process finishes, i.e., after x seconds from the beginning of the cooking process.

I EGO-CoMIST Prompts: Stage 1

Here we provide the prompts used in stage 1 (Counterfactual Instruction and Feedback Generation) of the EGO-CoMIST data generation pipeline.

I.1 Measurement error

As described in the main paper, given a recipe step: [org_recipe_step], we first ask the LLM to extract the attributes: quantity [quantity] and the corresponding ingredient [ingredient].

Extract Attributes from Recipe Step

You are an expert cooking assistant. You are watching a person follow a step by step recipe.

##INSTRUCTIONS:

The person is following the recipe step: [org_recipe_step].

Does this recipe step involve measuring a specific quantity or amount of an ingredient?

Examples include specific quantities like teaspoon, teaspoons, tablespoons, liters, ounces, grams, kgs, kilograms, cups, pints, quarts, gallons etc.

Extract the explicitly stated numerical quantity or amount and Answer 'YES' or 'NO'. If 'YES', then return the specific quantity and ingredient. Return a dict with two fields: {'ingredient': ..., 'quantity': ...}. DO NOT RETURN ANYTHING ELSE.

Then, we ask the LLM to create a counterfactual quantity [counterfactual_quantity], reasonable for the given recipe step. We use different prompts for greater and smaller quantities, which ask the LLM to generate greater or smaller counterfactual quantities respectively, as shown below.

Counterfactual Attributes (Greater)

You are an expert cooking assistant.

##INSTRUCTIONS:

The following recipe step: [org_recipe_step]; uses the following quantity: [quantity] of the item: [ingredient]. Suggest an alternative reasonable quantity of the item greater than [quantity] for the recipe step. RETURN JUST THE QUANTITY. DO NOT RETURN ANYTHING ELSE.

Based on the original and counterfactual quantities, we generate the counterfactual instruction and feedback pair.

Counterfactual Instruction

You are an expert cooking assistant.

##INSTRUCTIONS:

Modify the instruction [org_instruction] for the recipe step: [org_recipe_step]; such that the recipe step uses [counterfactual_quantity] of [ingredient]. RETURN JUST THE INSTRUCTION DO NOT RETURN ANYTHING ELSE.

Counterfactual Feedback

You are an expert cooking assistant. You are assisting a person step by step through a recipe.

##INSTRUCTIONS:

Instead of using the specified quantity: [quantity] of [ingredient], the person instead used the following quantity: [counterfactual_quantity]. Given the correct specified quantity and the incorrectly used quantity provide an appropriate feedback message. The feedback message should be short and one line in length. Make sure to point out the mistake clearly in the feedback message in a single line. RETURN JUST THE FEEDBACK MESSAGE DO NOT RETURN ANYTHING ELSE.

I.2 Preparation error

As described in the main paper, given a recipe step [org_recipe_step], we ask the LLM to extract the preparation method [current_prep_method] and the object of the preparation method [object] and then propose a counterfactual (alternative) preparation method [alternative_prep_method] for the object that aligns with the original recipe step. Note, that in the prompt we use “alternative” instead of “counterfactual” for clarity.

Extract Attributes and Propose Counterfactual Attributes

You are an expert cooking assistant.

##INSTRUCTIONS:

Given a recipe step, your task is to propose variants of that recipe step that use an alternative preparation method. Alternative preparation methods include: different blending/whisking/mixing/beating methods; using different utensils or ingredients; using left instead of right hands and vice versa; dealing with fluids differently; cutting or chopping ingredients differently.

Now, given the following recipe step: [org_recipe_step]. Extract the specific preparation method used, the food item or cooking utensil used (object) and then propose an alternative preparation method. Make sure that the proposed alternative preparation method is realistic and likely to be provided by a expert cooking assistant.

Return a dict with five keys: {'current_prep_method': ..., 'object': ..., 'alternative_prep_method': ..., 'current_prep_method_duration': ..., 'alternative_prep_method_duration': ...}. Where, if the current preparation method (current_prep_method) requires cooking (e.g. heating, boiling, microwaving) for a specific duration this duration is returned in [current_prep_method_duration] field (None otherwise); and if the alternative preparation method (alternative_prep_method) requires cooking (e.g. heating, boiling, microwaving) for a specific duration this duration is returned in [alternative_prep_method_duration] field (None otherwise). Keep the [alternative_prep_method] in the dict very short. RETURN JUST THE COOKING PREP METHOD DICT DO NOT RETURN ANYTHING ELSE.

Based on the original and counterfactual preparation methods, we generate the counterfactual instruction and feedback pair.

Counterfactual Instruction

You are an expert cooking assistant.

##INSTRUCTIONS:

Generate a cooking instruction that uses the following preparation method: [alternative_prep_method] on: [object] for: [alternative_prep_method_duration]. Here is an example of a similar instruction: [org_instruction]. RETURN JUST THE INSTRUCTION DO NOT RETURN ANYTHING ELSE.

Counterfactual Feedback

You are an expert cooking assistant.

##INSTRUCTIONS:

Instead of the correct preparation method: [org_prep_method] the person used the preparation method: [alternative_prep_method] – on the food item or cooking utensil: [object]. Provide an appropriate feedback message to the person. The feedback message should be one line in length and clearly point out the mistake made by the person. Make sure to point out the mistake clearly in the feedback message in a single line. RETURN JUST THE FEEDBACK MESSAGE DO NOT RETURN ANYTHING ELSE.

I.3 Technique error

As described in the main paper, given a recipe step [org_recipe_step], we ask the LLM to extract the specific cooking technique used [technique] along with the ingredient or item (object) [item] used in the specific technique.

Extract Attributes from Recipe Step

You are an expert cooking assistant. You are assisting a person step by step through a recipe.

##INSTRUCTIONS:

The person is trying to complete the following recipe step: [org_recipe_step].

If the recipe step uses a specific cooking technique, example techniques include: roll, pat, squeeze, hold, measuring, adding, transferring, chopping, cutting, peeling, spiralizing, flipping, stirring, whisking, beating, return the cooking technique, return a dict with the cooking technique and the food item or ingredient that it is used on: { 'technique': ..., 'item':... }. If not, return None. DO NOT RETURN ANYTHING ELSE.

Then, we ask the LLM to propose a counterfactual technique [counterfactual_technique] that aligns with the original recipe step. In the prompt, again we use “alternative” instead of “counterfactual” for clarity.

Counterfactual Attributes

You are an expert cooking assistant.

##INSTRUCTIONS:

Given a recipe step, [org_recipe_step] that uses the cooking technique: [technique] on [item], your task is to propose an alternative cooking technique. Make sure that the proposed alternative cooking technique is realistic given the recipe step. RETURN JUST THE ALTERNATIVE COOKING TECHNIQUE DO NOT RETURN ANYTHING ELSE.

Based on the original and counterfactual techniques, we generate the counterfactual instruction and feedback pair.

Counterfactual Instruction

You are an expert cooking assistant.

##INSTRUCTIONS:

Given the cooking instruction [org_instruction], generate a new instruction that uses the following cooking technique instead: [alternative_technique]. RETURN JUST THE INSTRUCTION DO NOT RETURN ANYTHING ELSE.

Counterfactual Feedback

You are an expert cooking assistant.

##INSTRUCTIONS:

Instead of the correct technique: [counterfactual_technique] the person used the technique: [technique] – on the: [item]. Provide an appropriate feedback message to the person. The feedback message should be one line in length and clearly point out the mistake made by the person. Make sure to point out the mistake clearly in the feedback message in a single line. RETURN JUST THE FEEDBACK MESSAGE DO NOT RETURN ANYTHING ELSE.

I.4 Temperature error

As described in the main paper, given a recipe step [org_recipe_step], we first ask the LLM to extract the cooking temperature or heat setting [heat_setting] using the kitchen appliance or utensil [kitchen_item] used on the [food_item].

Extract Attributes from Recipe Step

You are an expert cooking assistant. You are assisting a person step by step through a recipe.

##INSTRUCTIONS:

The person is following the recipe step: [org_recipe_step]

Does this recipe step involve cooking an ingredient or a food item on a kitchen appliance or utensil at a specific temperature or heat setting? Examples include setting an oven or a stove at a specific temperature or heat setting. If yes, then return the specific kitchen appliance or utensil (kitchen item), temperature or heat setting (heat setting) and the food item or ingredient (food item). Return a dict with three fields: {'kitchen_item': ..., 'heat_setting': ..., 'food_item': ...} else return None. DO NOT RETURN ANYTHING ELSE.

Then, we ask the LLM to suggest a counterfactual temperature or a heat setting [counterfactual_heat_setting]. We use separate (similar) prompts for higher or lower heat settings. In the prompt, we again use “alternative” instead of “counterfactual” for clarity.

Counterfactual Attribute (Higher)

You are an expert cooking assistant.

##INSTRUCTIONS:

The following recipe step: [org_recipe_step]; uses the following kitchen appliance/utensil: [kitchen_item]; to cook the following food item: [food_item]; at the following heat setting: [heat_setting]. Suggest an alternative reasonable heat setting higher than [heat_setting] for the recipe step. RETURN JUST THE HEAT SETTING IF POSSIBLE ELSE RETURN NONE. DO NOT RETURN ANYTHING ELSE.

Based on the original and counterfactual heat settings, we generate the counterfactual instruction and feedback pair.

Counterfactual Instruction

You are an expert cooking assistant.

##INSTRUCTIONS:

Modify the cooking instruction: [org_instruction]; such that the recipe step uses the following heat setting: [counterfactual_heat_setting] – while cooking [food_item] on the utensil/appliance: [kitchen_item]. RETURN JUST THE INSTRUCTION DO NOT RETURN ANYTHING ELSE.

Counterfactual Feedback

You are an expert cooking assistant.

##INSTRUCTIONS:

Instead of using the correct heat setting: [heat_setting], the person instead used a incorrect heat setting: [counterfactual_heat_setting] – on the food item: [food_item] – and using the kitchen utensil/appliance: [kitchen_item].

Provide an appropriate feedback message. The feedback message should be short and one line in length. Make sure to point out the heat setting clearly in a feedback message in a single line. RETURN JUST THE FEEDBACK MESSAGE DO NOT RETURN ANYTHING ELSE.

I.5 Timing error

As described in the main paper, given a recipe step [org_recipe_step], we first ask the LLM to extract the duration [amount_of_time], the food item or ingredient [food_item] and the specific kitchen appliance or utensil [kitchen_item] the [food_item] is cooked in.

Extract Attributes from Recipe Step

You are an expert cooking assistant. You are assisting a person step by step through a recipe.

##INSTRUCTIONS:

The person is following the recipe step: [org_recipe_step]. Does this recipe step involve cooking an ingredient or a food item on a kitchen appliance or utensil for a specific amount of time? Examples include setting an oven or a stove at a specific temperature or heat setting.

If yes, then return the specific kitchen appliance or utensil (kitchen_item), the amount of time (amount_of_time) and the food item or ingredient (food_item). Return a dict with three fields: {'kitchen_item': ..., 'amount_of_time': ..., 'food_item': ...}. Else return None. DO NOT RETURN ANYTHING ELSE.

Then, we ask the LLM to propose a counterfactual reasonable amount of time [counterfactual_amount_of_time]. In the prompt, we again use “alternative” instead of “counterfactual” for clarity.

Counterfactual Attribute (Greater)

You are an expert cooking assistant.

##INSTRUCTIONS:

The following recipe step: [org_recipe_step]; uses the following kitchen appliance/utensil: [kitchen_item]; to the cook the following food item: [food_item]; for the following amount of time: [amount_of_time]. Suggest an alternative reasonable amount of time significantly greater (at least 15 seconds) than [amount_of_time] for the recipe step. RETURN JUST THE AMOUNT OF TIME IF POSSIBLE ELSE RETURN NONE. DO NOT RETURN ANYTHING ELSE.

Based on the original and counterfactual amounts of time, we generate the counterfactual instruction and feedback pair.

Counterfactual Instruction

You are an expert cooking assistant.

##INSTRUCTIONS:

Modify the cooking instruction: [org_instruction]; such that the recipe step uses the following amount of time: [counterfactual_amount_of_time] – while cooking [food_item] on the utensil/appliance: [kitchen_item]. RETURN JUST THE INSTRUCTION DO NOT RETURN ANYTHING ELSE.

Counterfactual Feedback (Greater)

You are an expert cooking assistant. You are assisting a person step by step through a recipe.

##INSTRUCTIONS:

Instead of the using the correct amount of time for cooking: [amount_of_time], the person did not cook the food item: [food_item] – and using the kitchen utensil/appliance: [kitchen_item] – for long enough. Provide an appropriate feedback message. The feedback message should be short and one line in length. Make sure to point out that the person did not cook for long enough clearly in the feedback message in a single line. RETURN JUST THE FEEDBACK MESSAGE DO NOT RETURN ANYTHING ELSE.

J EGO-COMIST Prompts: Stage 2

Here we provide the prompts used in the stage 2 (Counterfactual Feedback Timestamp Inference) of the EGO-COMIST data generation pipeline. The prompts always use the step by step descriptions [step_by_step_descriptions] extracted using the state of the art video LLM at timestamps [timestamps] that are 5 seconds apart using a sliding window, as described in the main paper.

J.1 Measurement error

In addition to the step by step descriptions [step_by_step_descriptions] at timestamps [timestamps], we use the attributes [quantity], [ingredient] and the counterfactual quantity [counterfactual_quantity] generated in the first stage of our EGO-COMIST data generation pipeline to infer the appropriate feedback timestamp.

Feedback Timestamp (Greater Counterfactual Quantity)

You are an expert cooking assistant. You are watching a person follow a step by step recipe.

##INSTRUCTIONS:

You have provided the person with the following instruction: [org_instruction].

You are now provided with the following step by step account of the person's activities, along with the corresponding timestamps: [step_by_step_instruction].

Your task is to find the timestamp at which it becomes apparent that the person uses less than the instructed quantity of the following: [ingredient]. The person was instructed to use the following quantity [quantity] but instead used [counterfactual_quantity].

WAIT and make sure that the person does not intend to use more (the correct quantity).

Choose one of the following options: [timestamps]. Return the timestamp in seconds from the options above as a python dict of the form: {'timestamp': ...}. DO NOT RETURN ANYTHING OTHER THAN THE PYTHON DICT.

J.2 Preparation error

In addition to the step by step descriptions [step_by_step_descriptions] at timestamps [timestamps], we use the attributes: [current_prep_method], [object] extracted from the recipe step and the counterfactual preparation method [alternative_prep_method] generated in the first stage of our EGO-COMIST data generation pipeline.

Feedback Timestamp

You are an expert cooking assistant. You are watching a person follow a step by step recipe.

##INSTRUCTIONS:

You have provided the person with the following instruction: [org_instruction].

You are now provided with the following step by step account of the person's activities, along with the corresponding timestamps: [step_by_step_instruction].

Your task is to find the timestamp at which it becomes apparent that the person has used the following (wrong) preparation method: [current_prep_method] – on the food item or cooking utensil: [object] – instead of the correct preparation method: [alternative_prep_method].

Choose one of the following options: [timestamps]. Return the timestamp in seconds from the options above as a python dict of the form: {'timestamp': ...}. DO NOT RETURN ANYTHING OTHER THAN THE PYTHON DICT.

J.3 Technique error

In addition to the step by step descriptions [step_by_step_descriptions] at timestamps [timestamps], we use the attributes: [technique] and [item], extracted from the recipe step and the counterfactual technique [alternative_technique] generated in the first stage of our EGO-COMIST data generation pipeline.

Feedback Timestamp

You are an expert cooking assistant. You are watching a person follow a step by step recipe.

##INSTRUCTIONS:

You have provided the person with the following instruction: [org_instruction].

You are now provided with the following step by step account of the person's activities, along with the corresponding timestamps: [step_by_step_instruction].

Your task is to find the timestamp at which it becomes apparent that the person is using the wrong technique: [technique] – and the person does not intend to use the correct technique: [alternative_technique].

Choose one of the following options: [timestamps]. Return the timestamp in seconds from the options above as a python dict of the form: {'timestamp': ...}. DO NOT RETURN ANYTHING OTHER THAN THE PYTHON DICT.

J.4 Temperature error

In addition to the step by step descriptions [step_by_step_descriptions] at timestamps [timestamps], we use the attributes: [heat_setting], [kitchen_item], [food_item] extracted from the recipe step and the counterfactual heat setting [counterfactual_heat_setting] generated in the first stage of our EGO-COMIST data generation pipeline.

Feedback Timestamp

You are an expert cooking assistant. You are watching a person follow a step by step recipe.

##INSTRUCTIONS:

You have provided the person with the following instruction: [org_instruction].

You are now provided with the following step by step account of the person's activities, along with the corresponding timestamps: [step_by_step_instruction].

Your task is to find the timestamp at which it becomes apparent that the person uses the wrong heat/temperature setting: [heat_setting] – of the utensil/appliance: [kitchen_item] – to cook the following: [food_item]– instead of the correct heat/temperature setting: [counterfactual_heat_setting].

Choose the EARLIEST timestamp where it becomes clear that the person has chosen the incorrect heat/temperature setting. (This is usually when the person starts an appliance or adjusts the heat setting).

Choose one of the following options: [timestamps]. Return the timestamp in seconds from the options above as a python dict of the form: {'timestamp': ...}. DO NOT RETURN ANYTHING OTHER THAN THE PYTHON DICT.

J.5 Timing error

In addition to the step by step descriptions [step_by_step_descriptions] at timestamps [timestamps], we use the attributes: [amount_of_time] and [kitchen_item] extracted from the recipe step and the counterfactual amount of time [counterfactual_amount_of_time] generated in the first stage of our EGO-COMIST data generation pipeline. We use two different prompts for [counterfactual_amount_of_time] being larger or smaller than [amount_of_time] in the original recipe step.

Feedback Timestamp (Smaller Counterfactual Duration)

You are an expert cooking assistant. You are watching a person follow a step by step recipe.

##INSTRUCTIONS:

You have provided the person with the following instruction: [org_instruction].

You are now provided with the following step by step account of the person's activities, along with the corresponding timestamps: [step_by_step_instruction].

First find the [start_timestamp] when the person starts to cook: [food_item] – using the utensil/appliance: [kitchen_item]. Now, instead of the specified amount of time: [amount_of_time] – the person cooks for: [counterfactual_amount_of_time]. Find the timestamp [critical_timestamp] at which it becomes apparent that the person does not cook: [food_item] – using the utensil/appliance: [kitchen_item] – for long enough.

Choose one of the following options: [timestamps]. Choose the LAST timestamp where it becomes clear that the person has not cooked for long enough, after the person stops cooking [critical_timestamp] with the [kitchen_item], at least [amount_of_time] from the beginning of the cooking process [start_timestamp].

Return the timestamps in seconds from the options above as a python dict of the form: {'start_timestamp': ..., 'critical_timestamp': ...}. The 'start_timestamp' and 'critical_timestamp' should be at least [amount_of_time] seconds apart. If you are not sure of the [start_timestamp] or [critical_timestamp] return None. DO NOT RETURN ANYTHING OTHER THAN THE PYTHON DICT.

Feedback Timestamp (Larger Counterfactual Duration)

You are an expert cooking assistant. You are watching a person follow a step by step recipe.

##INSTRUCTIONS:

You have provided the person with the following instruction: [org_instruction].

You are now provided with the following step by step account of the person's activities, along with the corresponding timestamps: [step_by_step_instruction].

First find the [start_timestamp] when the person starts to cook: [food_item] – using the utensil/appliance: [kitchen_item]. Now, instead of the specified amount of time: [amount_of_time] – the person cooks for: [counterfactual_amount_of_time]. Find the timestamp [critical_timestamp] at which it becomes apparent that the person cooks: [food_item] – using the utensil/appliance: [kitchen_item] – for too long.

Choose one of the following options: [timestamps]. Choose the LAST timestamp where it becomes clear that the person has cooked for too long, about [amount_of_time] from the beginning of the cooking process [start_timestamp] with the [kitchen_item] (don't wait too long).

Return the timestamps in seconds from the options above as a python dict of the form: {'start_timestamp': ..., 'critical_timestamp': ...}. The 'start_timestamp' and 'critical_timestamp' should be at least [amount_of_time] seconds apart. If you are not sure of the [start_timestamp] or [critical_timestamp] return None. DO NOT RETURN ANYTHING OTHER THAN THE PYTHON DICT.